CSE 242 Assignment 2

Your name: Nidhi Shah Student ID: nshah17

Question 1 (Maximum likelihood estiamtion and Posterior)

Assume we have a coin that has some unknown probability h of coming up heads (and probability $1-h$ of coming up tails). If the coin is flipped five times getting three heads and two tails ($\mathbf{HHHTT}$) then:

(a -- 10pts) What is the maximum likelihood estimate for h?

(b -- 10pts) Assume that (before flipping the coin) we have a prior density $p(h)$ for the various values of $h\in [0,1]$. Give the formula for the posterior probability density $p(h \mid \mathbf{HHHTT})$ as a function of the prior $p(h)$.

For Question 1(b), the solution is acceptable if the formula contains an integral.

Solution (a):

The likelihood of observing 3 heads and 2 tails is

$$ L(h)=\binom{5}{3}h^3(1-h)^2 $$$$ L(h)=\binom{5}{3}h^3(1-h)^2 $$

The maximum likelihood estimate is the observed proportion of heads:

$$ \hat{h}_{\mathrm{MLE}}=\frac{3}{5}=0.6 $$$$ \hat{h}_{\mathrm{MLE}}=\frac{3}{5}=0.6 $$

Solution (b):

By Bayes' rule, the posterior density is proportional to the likelihood times the prior. Therefore,

$$ p(h \mid \mathbf{HHHTT})=\frac{h^3(1-h)^2 p(h)}{\int_0^1 u^3(1-u)^2 p(u)\,du}, \quad 0 \le h \le 1 $$$$ p(h \mid \mathbf{HHHTT})=\frac{h^3(1-h)^2 p(h)}{\int_0^1 u^3(1-u)^2 p(u)\,du}, \quad 0 \le h \le 1 $$

The binomial coefficient $\binom{5}{3}$$\binom{5}{3}$ cancels between the numerator and denominator.

If you are not familair with Latex, you may attach a figure/screen-shoot and display the code below.

```
from IPython.display import Image
# Replace the figure name
# Image(filename='sample.png')
```

Question 2 (Linear Regression)

Reading data using Pandas

```
# Read the dataset you will be working on
# The dataframe loaded with pandas is named as data

import pandas as pd
data = pd.read_csv('/content/Advertising.csv', index_col=0)  # Modify

# Take a look at the first 5 rows
data.head()
```

	TV	Radio	Newspaper	Sales
1	230.1	37.8	69.2	22.1
2	44.5	39.3	45.1	10.4
3	17.2	45.9	69.3	9.3
4	151.5	41.3	58.5	18.5
5	180.8	10.8	58.4	12.9

Next steps: Generate code with data | New interactive sheet

```
data.shape
```

```
(200, 4)
```

Features and responses

What are the features?

- **TV:** advertising dollars spent on TV for a single product in a given market (in thousands of dollars)
- **Radio:** advertising dollars spent on Radio
- **Newspaper:** advertising dollars spent on Newspaper

What is the response?

- **Sales:** sales of a single product in a given market (in thousands of items)

What else do we know?

- There are 200 **observations**, and each observation (each row) is a single market.
- **Your main task**: predict the sales of a single product in a given market.
- Since the response variable is continuous, this is a **regression** problem.

Linear regression

Pros: fast, no tuning required, highly interpretable

Cons: adopting the linear regression is unlikely to always generate the best predictive accuracy, since Linear Regression presumes a linear relationship between the features and response. This assumption may not always make sense. (But if one can *transform* the features so that do have a linear relationship, sometimes progress can be made, for example by taking logs.)

Expression:

- $y$$y$: the response
- $\beta_0$$\beta_0$: the intercept
- $\beta_1$$\beta_1$: the coefficient for $x_1$$x_1$ (the first feature)
- $\beta_i$$\beta_i$ is the coefficient for $x_i$$x_i$ (the ith feature, $i\in \{1, 2, ..., n\}$$i\in \{1, 2, ..., n\}$)

For this regression task:

$\text{Sales} = \beta_0 + \beta_1 \times \text{TV} + \beta_2 \times \text{Radio} + \beta_3 \times \text{Newspaper}$$\text{Sales} = \beta_0 + \beta_1 \times \text{TV} + \beta_2 \times \text{Radio} + \beta_3 \times \text{Newspaper}$

The $\beta$$\beta$ values ($\beta=[\beta_0, \beta_1, \beta_2, \beta_3]$$\beta=[\beta_0, \beta_1, \beta_2, \beta_3]$) are called the **model coefficients**. These values are "learned" during the model fitting step using the "least squares" criterion. Then, we can make use of the fitted model to make predictions!

Prepare the dataset for training use

- Scikit-learn expects X (feature matrix) and y (response vector) to be NumPy arrays.
- Pandas is built on top of NumPy, and exposes numpy methods. Thus, X can be a pandas DataFrame, y can be a pandas Series!

```
# Prepare the feature X
X = data[['TV', 'Radio', 'Newspaper']]

# check the type and shape of X
print(type(X))
print(X.shape)
```

```
<class 'pandas.core.frame.DataFrame'>
(200, 3)
```

```
# Prepare the response Y
y = data['Sales']
# check the type and shape of y
print(type(y))
print(y.shape)
```

```
<class 'pandas.core.series.Series'>
(200,)
```

Question 2.1 (Split the dataset into train and test parts)

For features $X$$X$ and response $y$$y$, use **sklearn** to perform the the splitting of dataset: $80\%$$80\%$ for train data, $20\%$$20\%$ for the test data.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Double check the shape of train and test data:

```
print(X_train.shape)
print(y_train.shape)
print(X_test.shape)
print(y_test.shape)
```

```
(160, 3)
(160,)
(40, 3)
(40,)
```

Question 2.2 (Train a Linear regression model via Scikit-learn)

Assume your model is named as **model**, and you train the linear regression model to fit on the training data.

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)
```

```
▾ LinearRegression
LinearRegression()
```

```
# Take a look at your model coefficients
import numpy as np
print(np.round(model.intercept_, 4))
print([np.round(val, 4) for val in model.coef_])
```

```
2.9791
[np.float64(0.0447), np.float64(0.1892), np.float64(0.0028)]
```

```
# Match the feature names with the trained coefficients
list(zip(data.columns[:3].tolist(), model.coef_))
```

```
[('TV', np.float64(0.044729517468716326)),
 ('Radio', np.float64(0.18919505423437652)),
 ('Newspaper', np.float64(0.0027611143413671935))]
```

Question 2.3 (Interpreting model coefficients)

- What is the trained linear regression model?
- **(replace $\beta_i$$\beta_i$ below with the trained coefficients, reserve four decimal places, 5 pts)**

$$\text{Sales} = \beta_0 + \beta_1 \times \text{TV} + \beta_2 \times \text{Radio} + \beta_3 \times \text{Newspaper}$$$$\text{Sales} = \beta_0 + \beta_1 \times \text{TV} + \beta_2 \times \text{Radio} + \beta_3 \times \text{Newspaper}$$

- **(Open) question (5 pts)**: how do you interpret the coefficient of **TV** ($\beta_1$$\beta_1$)?

Your solution for Question 2.3:

The trained model is approximately

$$
\text{Sales}=2.9791+0.0447(\text{TV})+0.1892(\text{Radio})+0.0028(\text{Newspaper}). $$$$
\text{Sales}=2.9791+0.0447(\text{TV})+0.1892(\text{Radio})+0.0028(\text{Newspaper}). $$

The TV coefficient is approximately $0.0447$$0.0447$. Holding Radio and Newspaper spending constant, an additional $1,000 spent on TV advertising is associated with an estimated increase of about $0.0447$$0.0447$ thousand sales, or about 44.7 items, on average. This describes the model's association, not necessarily a causal effect.

Making predictions

```
# make predictions on the testing set
y_pred = model.predict(X_test)
# Print the test accuracy score from your trained linear regression m
model.score(X_test,y_test)
```

```
0.899438024100912
```

Model evaluation metrics for regression

Evaluation metrics for classification problems, such as **accuracy**, are not useful for regression problems. Instead, we need evaluation metrics designed for comparing continuous values. (Since predictions would hardly ever exactly match the measured quantities.)

Mean Squared Error (MSE) is the mean of the squared errors:

$$\frac 1n\sum_{i=1}^n(y_i-\hat{y}_i)^2$$$$\frac 1n\sum_{i=1}^n(y_i-\hat{y}_i)^2$$

Root Mean Squared Error (RMSE) is a popular evaluation metric for regression, which is defined as the square root of the mean of the squared errors:

$$\sqrt{\frac 1n\sum_{i=1}^n(y_i-\hat{y}_i)^2}$$$$\sqrt{\frac 1n\sum_{i=1}^n(y_i-\hat{y}_i)^2}$$

Question 2.4 (Evaluate your predictions on test features with RMSE metric)

Hint: you may calculate MSE using scikit-learn, but RMSE is a little bit different from MSE.

```
from sklearn import metrics

rmse = np.sqrt(metrics.mean_squared_error(y_test, y_pred))
print("Test RMSE:", round(rmse, 4))
```

```
Test RMSE: 1.7816
```

Question 2.5 (Does the feature 'Newspaper' help?)

```
X_two = data[['TV', 'Radio']]
X_two_train, X_two_test, y_two_train, y_two_test = train_test_split(
    X_two, y, test_size=0.2, random_state=42
)

model_two = LinearRegression()
model_two.fit(X_two_train, y_two_train)
y_two_pred = model_two.predict(X_two_test)
rmse_two = np.sqrt(metrics.mean_squared_error(y_two_test, y_two_pred))

print("RMSE with TV, Radio, and Newspaper:", round(rmse, 4))
print("RMSE with TV and Radio only:", round(rmse_two, 4))
```

```
RMSE with TV, Radio, and Newspaper: 1.7816
RMSE with TV and Radio only: 1.7714
```

Your observations:

With all three features, the test RMSE is about **1.7816**. With only TV and Radio, the test RMSE is about **1.7714**. Since a lower RMSE means smaller prediction errors, removing Newspaper slightly improves predictions on this test set. This suggests Newspaper does not help much here and may add a little noise, though the difference is small and results can vary with the split.

Question 3 (PCA)

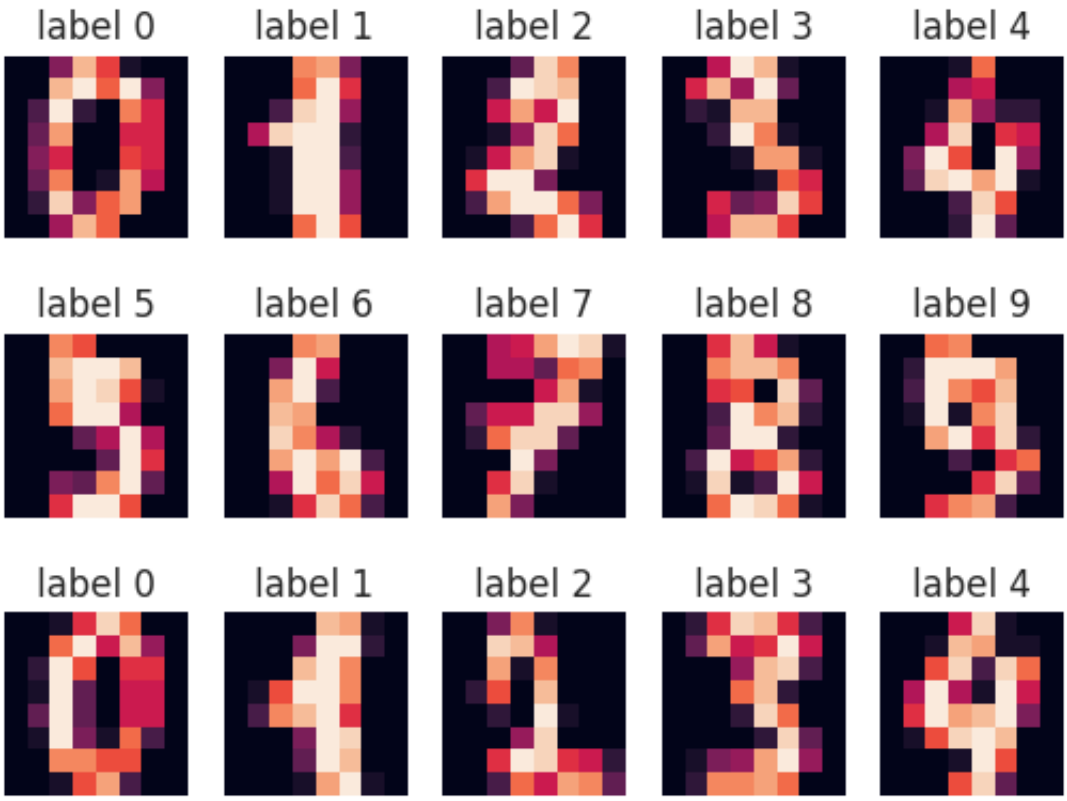
```
# import libraries
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
sns.set()
```

```
from sklearn.datasets import load_digits
data_digit = load_digits()
data_digit.data.shape
```

```
(1797, 64)
```

As you have seen above, there are 1797 rows and 64 columns in this dataset. Each row denotes the pixel values of an image. Since each image is 8*8 (in pixel), the dimension of each feature is 64. Let's take a look at a few images contained in this dataset.

```
# A help function to display images with labels
def display_digits(imgs):
    h = 3
    w = 5
    fig, axes = plt.subplots(h, w)
    for i in range(h):
        for j in range(w):
            if imgs.shape[1] == 64:
                axes[i, j].imshow(imgs[i * w + j].reshape(8, 8))
            else:
                axes[i, j].imshow(imgs[i * w + j])
            axes[i, j].axis('off')
            axes[i, j].set_title(f'label {data_digit.target[i * w + j]
# Take a look at a few examples
display_digits(data_digit.images)
```



PCA finds uncorrelated orthogonal axes (principal components) in a high dimensional space (i.e., 64) to project the feature onto principal components.

Question 3.1 (PCA and dimensionality deduction with Scikit-learn)

Fit the model with feature X (**data_digit.data** for this dataset) and apply the dimensionality reduction on X, keep 2 components.

```
from sklearn.decomposition import PCA


pca_2 = PCA(n_components=2)
projected = pca_2.fit_transform(data_digit.data)

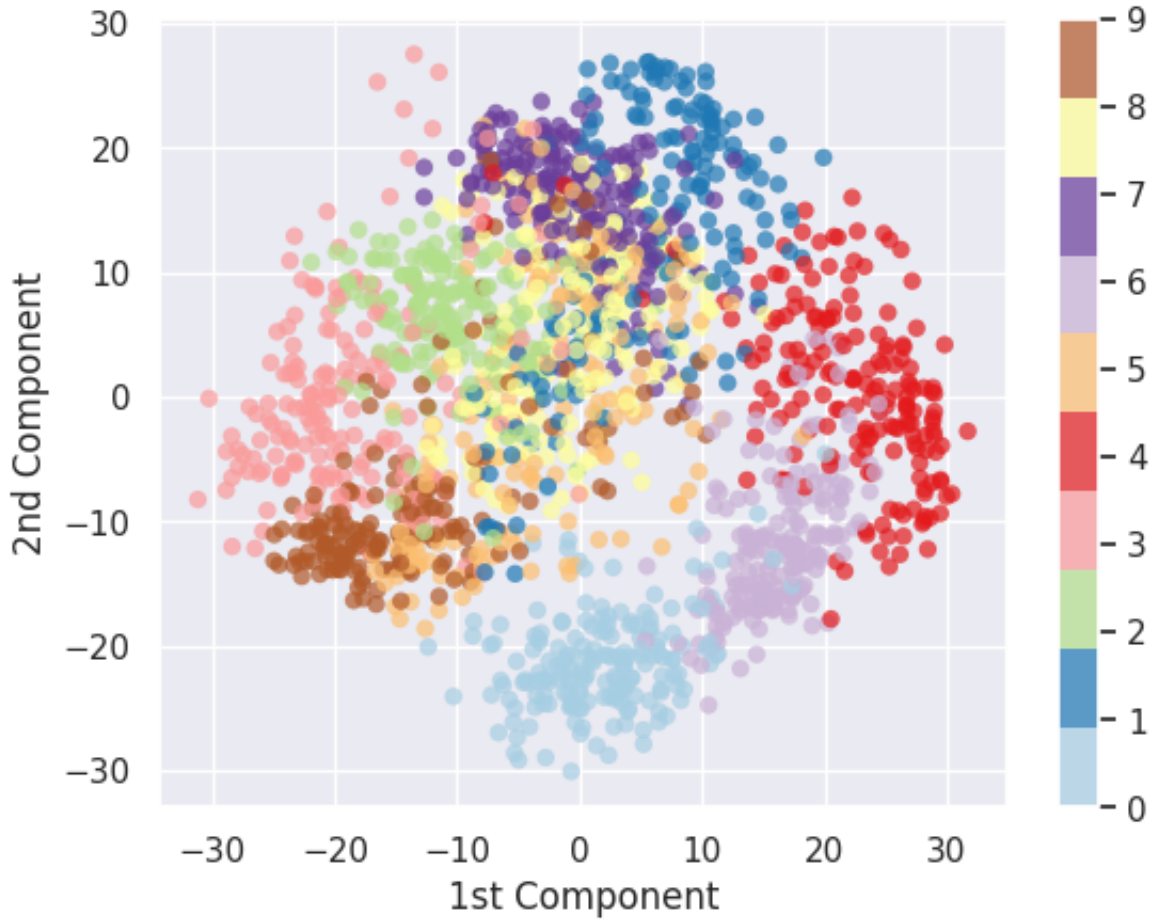

# The code below checks whether the feature dimension 64 is reduced t
print(data_digit.data.shape)
print(projected.shape)
```

```
(1797, 64)
(1797, 2)
```

With the reduced dimension of features, we can now visualize the projected/reduced features in a 2-dimension plot as below. There are hardly any overlaps between digit 0 and any other digits. While in the central area of the figure, features are not well separable.

```
plt.scatter(projected[:, 0], projected[:, 1],
            c=data_digit.target, edgecolor='none', alpha=0.7,
            cmap=plt.cm.get_cmap('Paired', 10))
plt.xlabel('1st Component')
plt.ylabel('2nd Component')
plt.colorbar()
plt.figure(figsize=(18, 8))
```

```
/tmp/ipykernel_1896/284023998.py:3: MatplotlibDeprecationWarning: The g
  cmap=plt.cm.get_cmap('Paired', 10))
<Figure size 1800x800 with 0 Axes>
```




```
<Figure size 1800x800 with 0 Axes>
```

Question 3.2 (How many components do we need?)

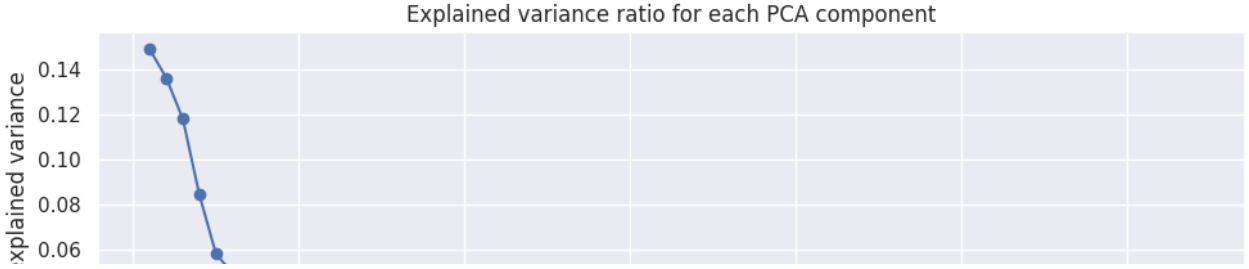
Print the ratio of variance explained by each of the selected components;

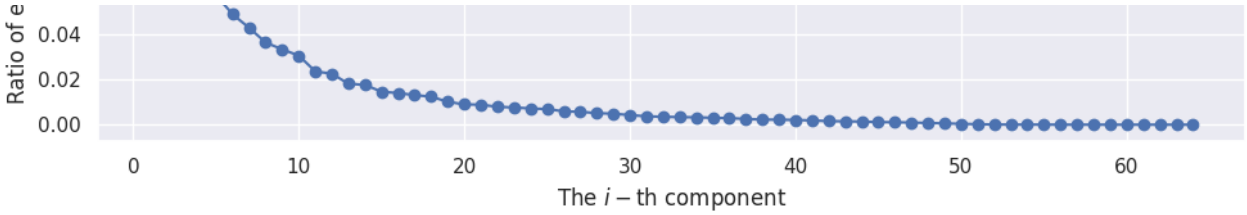
Visualize with matplotlib or seaborn (x axis: $i-th$$i-th$ component; y axis: the corresponding ratio of explained variance).

```
pca_all = PCA()
pca_all.fit(data_digit.data)
variance_ratios = pca_all.explained_variance_ratio_
print(variance_ratios)

plt.figure(figsize=(10, 4))
plt.plot(range(1, len(variance_ratios) + 1), variance_ratios, marker=
plt.xlabel('The $i-$th component')
plt.ylabel('Ratio of explained variance')
plt.title('Explained variance ratio for each PCA component')
plt.tight_layout()
```

```
[1.48905936e-01 1.36187712e-01 1.17945938e-01 8.40997942e-02
 5.78241466e-02 4.91691032e-02 4.31598701e-02 3.66137258e-02
 3.35324810e-02 3.07880621e-02 2.37234084e-02 2.27269657e-02
 1.82186331e-02 1.77385494e-02 1.46710109e-02 1.40971560e-02
 1.31858920e-02 1.24813782e-02 1.01771796e-02 9.05617439e-03
 8.89538461e-03 7.97123157e-03 7.67493255e-03 7.22903569e-03
 6.95888851e-03 5.96081458e-03 5.75614688e-03 5.15157582e-03
 4.89539777e-03 4.28887968e-03 3.73606048e-03 3.53274223e-03
 3.36683986e-03 3.28029851e-03 3.08320884e-03 2.93778629e-03
 2.56588609e-03 2.27742397e-03 2.22277922e-03 2.11430393e-03
 1.89909062e-03 1.58652907e-03 1.51159934e-03 1.40578764e-03
 1.16622290e-03 1.07492521e-03 9.64053065e-04 7.74630271e-04
 5.57211553e-04 4.04330693e-04 2.09916327e-04 8.24797098e-05
 5.25149980e-05 5.05243719e-05 3.29961363e-05 1.24365445e-05
 7.04827911e-06 3.01432139e-06 1.06230800e-06 5.50074587e-07
 3.42905702e-07 8.47114483e-19 5.53536763e-19 0.00000000e+00]
```





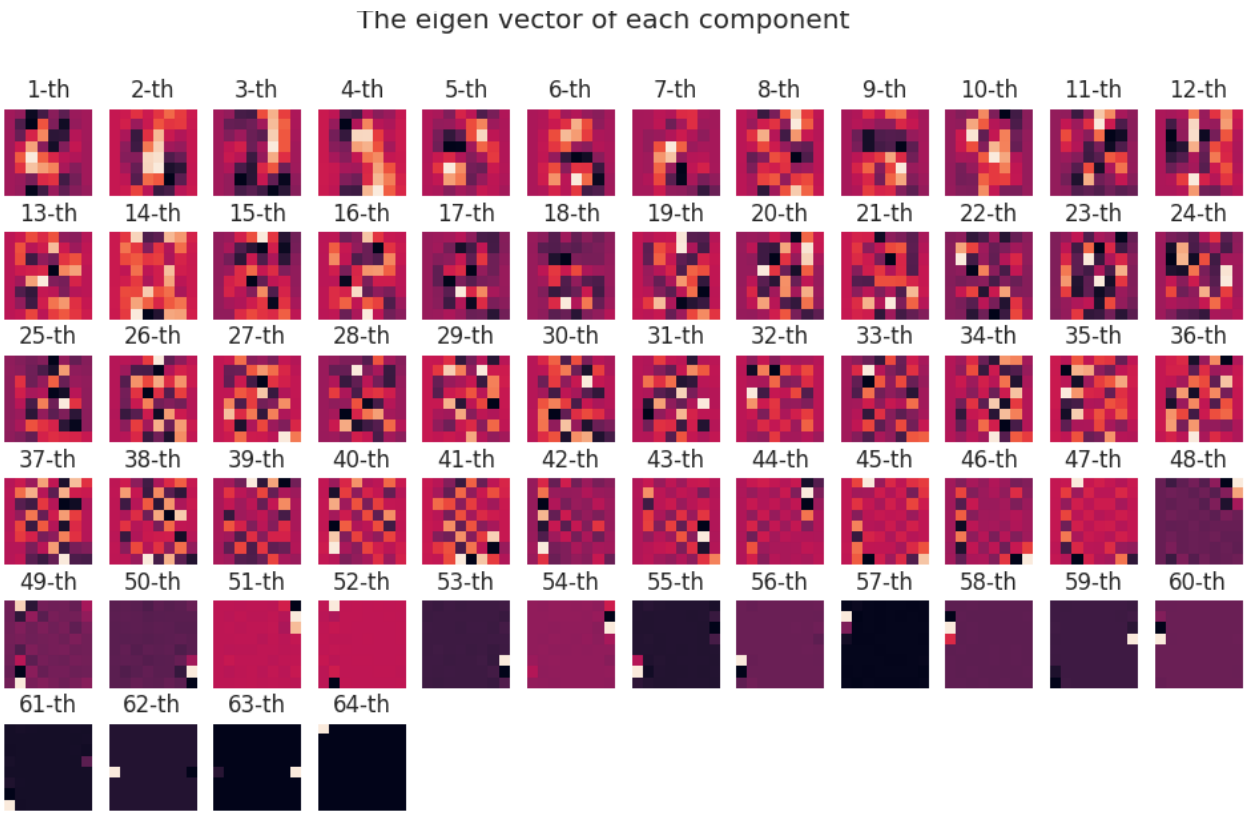
How to obtain eigen values and vectors from sklearn PCA

- Eigen values -- `your-PCA-model.explained_variance_`
- Eigen vectors -- `your-PCA-model.components_`

Eigen vectors (Eigen digits)

```
# Default # of components is the dimension of feature space
pca = PCA()
pca.fit(data_digit.data)
h = 6
w = 12
fig, axes = plt.subplots(h, w)
fig.set_size_inches(12, 7)
for i in range(h):
    for j in range(w):
        idx = i * w + j
        if idx > 63:
            axes[i, j].axis('off')
        else:
            axes[i, j].imshow(pca.components_[idx].reshape(8, 8))
            axes[i, j].axis('off')
            axes[i, j].set_title(f'{idx + 1}-th')
fig.suptitle('The eigen vector of each component')
```

```
Text(0.5, 0.98, 'The eigen vector of each component')
```



Question 3.3 What are your observations from the visualized eigen vectors?

Your solutions for Question 3.3:

The eigenvectors (principal components) look like faint, digit-shaped patterns rather than ordinary digit images. Each component represents a direction of variation across the pixel values. The first components capture broad, common patterns in the handwritten digits, while later components capture finer details and smaller variations. Some patterns are hard to interpret visually because a principal component can contain both positive and negative pixel weights.

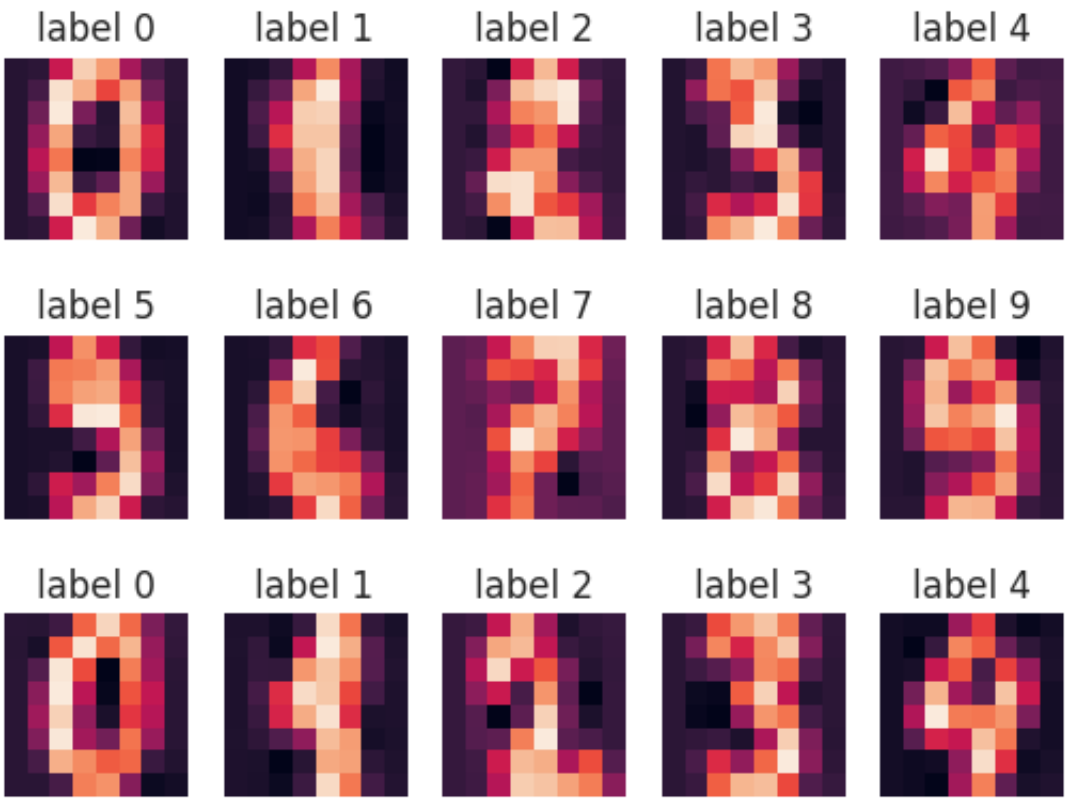
Feature reconstruction:

Suppose you only want to preserve $80\%$$80\%$ variance after applying PCA for the dimension reduction, due to storage issues, etc.

```
# Use pca from sklearn to fit on the feature space
pca_08 = PCA(0.8).fit(data_digit.data)

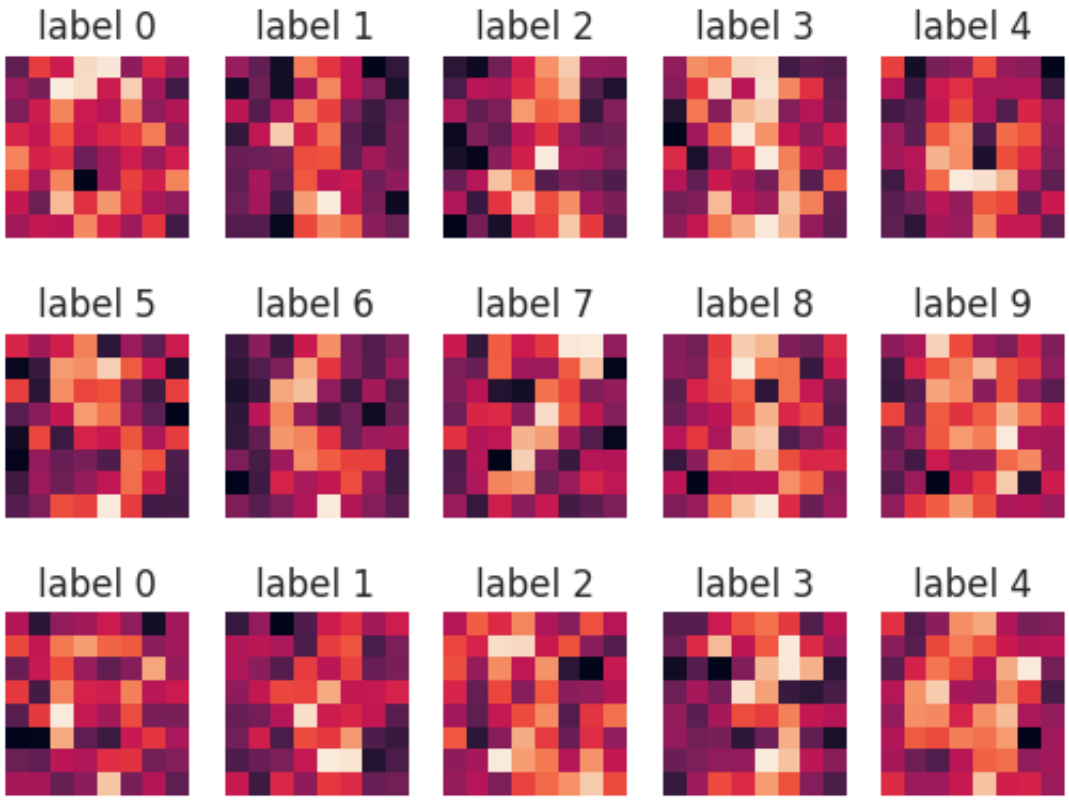
# Reconstruct
lower_dimension = pca_08.fit_transform(data_digit.data)
reconstructed = pca_08.inverse_transform(lower_dimension)

# Display the reconstructed images (with a much lower dimension)
display_digits(reconstructed)
```



Assume now you are given only a set of perturbed images (with some random noise). As shown below, the figures become much more difficult to recognize.

```
# Apply random noise on clean images
noisy_imgs = np.random.normal(data_digit.data, 5)
# Display the noisy images
display_digits(noisy_imgs)
```



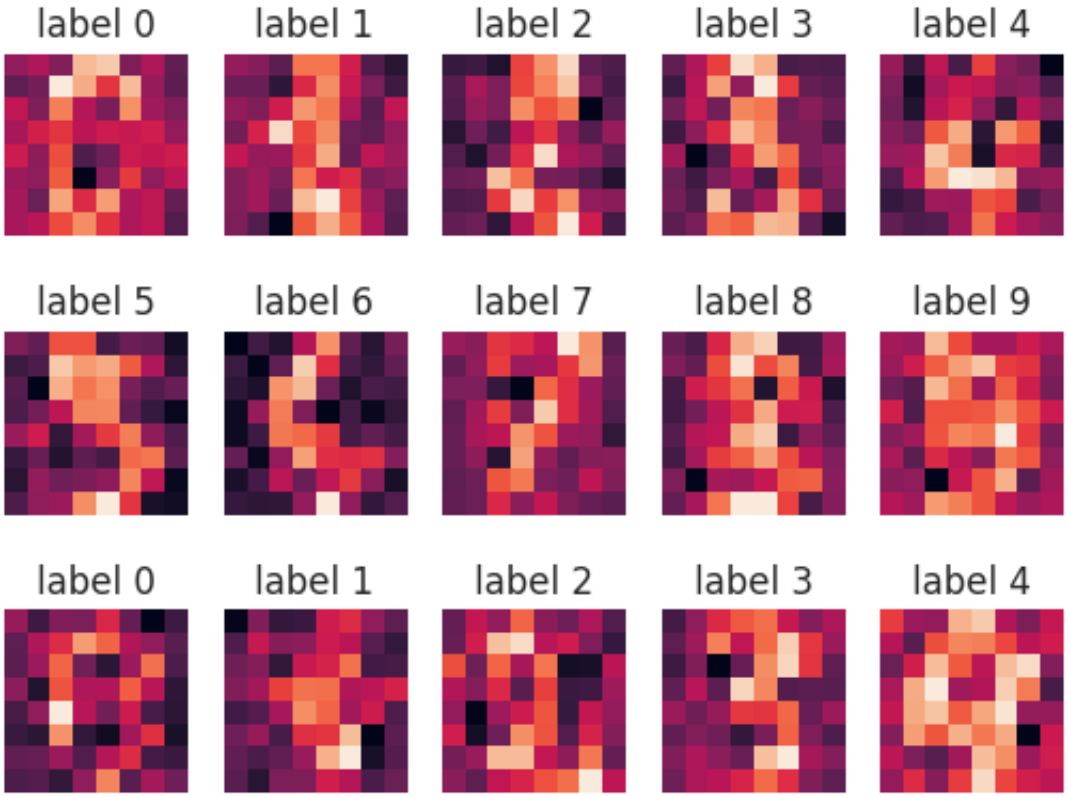
Question 3.4 Does PCA mitigate the random noise?

- Perform feature reconstruction on `noisy_imgs` (preserving $80\%$$80\%$ variance after applying PCA for the dimension reduction)
- Explain your understandings on why PCA reconstructs cleaner images.

```
# Fit PCA on the noisy images, keeping enough components to preserve 8
pca_noise = PCA(n_components=0.8)
lower_dimension_noise = pca_noise.fit_transform(noisy_imgs)
reconstructed_noise = pca_noise.inverse_transform(lower_dimension_noi

print("Number of components used:", pca_noise.n_components_)
# Display the reconstructed images
display_digits(reconstructed_noise)
```

```
Number of components used: 40
```



Your written answers for Question 3.4:

PCA helps clean up random noise because it compresses each image to its core component focusing on the big pattern which is shared across the dataset. When the images are reconstructed, only the top components are used and the small and irregular variations in it get left behind in turn making the digits look sharper. But it can't catch every bit of noise and some genuine details in the image can be lost along the way.